Let's go deep! I'll build this as a layered teaching session — concept first, then mechanics, then live interview drills.

Why chunking is the most underrated problem in RAG

People obsess over which LLM to use, which vector DB to pick. But in practice, the single biggest lever on RAG quality is **how you chunk your documents**. Bad chunking breaks retrieval before any other part of the pipeline even gets a chance.

The mental model: imagine shredding a book and trying to answer questions from the shreds. Too small = isolated words. Too large = irrelevant noise alongside the answer. The right chunk is self-contained unit of meaning. That's what every strategy below is trying to achieve.

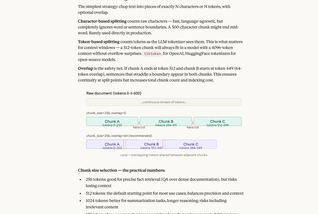
1. Fixed-Size Chunking

The simplest strategy: chop text into pieces of exactly N characters or N tokens, with optional overlap.

Character-based splitting counts raw characters — fast, language-agnostic, but completely ignores word or sentence boundaries. A 500-character chunk might end mid-word. Rarely used directly in production.

Token-based splitting counts tokens as the LLM tokenizer sees them. This is what matters for context windows — a 512-token chunk will always fit in a model with a 8096-token context without overflow surprises. Use `CharacterTextSplitter.from_tiktoken_encoder()` for OpenAI models.

Overlap is the safety net. If chunk A ends at token 512 and chunk B starts at token 449 (64-token overlap), sentences that straddle a boundary appear in both chunks. This ensures continuity at split points but increases total chunk count and indexing cost.



Chunk size selection — the practical rule:

- 256 tokens good for precise fact retrieval (QA over dense documentation), but risks losing context
- 512 tokens the default starting point for most use cases, balances precision and context
- 1024 tokens better for summarization tasks, longer reasoning chunks including irrelevant content
- 128 tokens for very granular passage retrieval or when using a parent-child strategy

The right chunk size depends on your document type and query style. Dense technical docs = smaller chunks. Narrative prose = larger. Always benchmark on your own data — never accept defaults blindly.

Pros of fixed-size chunking: predictable costs (you know exactly how many chunks per document), fast to implement, easy to reason about.

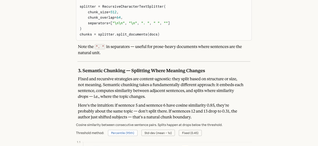
Cons: completely ignores document semantics. A 512-token chunk boundary could land mid-sentence, mid-table, or mid-code block. Retrieval quality suffers when the split happens at a meaningful boundary.

2. Recursive Splitting — The Production Default

`RecursiveCharacterTextSplitter` is the most commonly used chunker in LangChain and for good reason. Instead of blindly cutting at N characters, it tries a hierarchy of separators to find the most natural split point:

The separator hierarchy for English prose: `["\n\n", "\n", ". ", " "]`

It works like this: try to split on double newlines (paragraph boundaries) first. If a resulting piece is still too big, fall back to single newlines. Still too big? Try spaces (word boundaries). Last resort: raw characters.



This is why `RecursiveCharacterTextSplitter` is the default for most RAG systems — it respects document structure without needing to know anything about the document format. A paragraph boundary is almost always a better split point than mid-sentence.

In code:

```
python
from langchain_text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=64,
    separators=["\n\n", "\n", ". ", " "],
)
chunks = splitter.split_documents(docs)
```

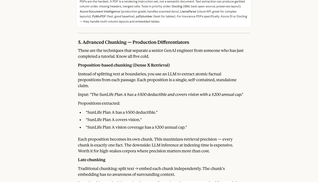
Note the `". "` in separators — useful for prose-heavy documents where sentences are the natural unit.

3. Semantic Chunking — Splitting Where Meaning Changes

Fixed and recursive strategies are content-agnostic: they split based on structure or size, not meaning. Semantic chunking takes a fundamentally different approach: it embeds each sentence, compares similarity between adjacent sentences, and splits where similarity drops — i.e., where the topic changes.

Here's the intuition: if sentence 3 and sentence 4 have cosine similarity 0.85, they're probably about the same topic — don't split them. If sentences 12 and 13 drop to 0.31, the author just shifted subjects — that's a natural chunk boundary.

Cosine similarity between consecutive sentences (gray line: breakpoint at 0.50 marks the threshold)



Toggle between the three threshold methods above to see how they affect where splits happen. This is exactly the decision you'd face when implementing semantic chunking.

The three threshold methods:

Percentile (e.g. 95 percentile of similarity scores): splits happens at the X lowest similarity points. Good because it adapts to the document's own distribution — if your document is internally very similar (like a technical manual), you need a lower absolute threshold than a general encyclopedia article.

Standard deviation (mean − 1σ): splits where similarity is more than one standard deviation below average. More principled statistically, self-normalizing across different documents.

Fixed threshold: simplest but requires calibration per corpus. 0.5–0.5 is typical. Breaks down on documents with very different internal similarity distributions.

LangChain's `SemanticChunker`:

```
python
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai.embeddings import OpenAIEmbeddings

chunker = SemanticChunker(
    OpenAIEmbeddings(),
    breakpoint_threshold_type="percentile",  # or "standard_deviation",
    "interquartile"
    breakpoint_threshold_amount=95
)
chunks = chunker.create_documents([text])
```

The hidden cost: semantic chunking calls the embedding model on every sentence during indexing. For a 10,000-document corpus, this is expensive. Cache aggressively, batch embed, and only use semantic chunking when fixed/recursive chunking measurably underperforms.

4. Document-Aware Chunking — Respecting Structure

Most real documents have structure that carries meaning: headers, sections, code blocks, tables. Generic splitters destroy this structure. Document-aware chunking preserves it.

5. Advanced Chunking — Production Differentiators

These are the techniques that separate a senior GenAI engineer from someone who has just completed a tutorial. Know all five cold.

Proposition-based chunking (Dense X Retrieval)

Instead of splitting text at boundaries, you use an LLM to extract atomic factual propositions from each passage. Each proposition is a single, self-contained, standalone claim.

Input: "The InstaLife Plan A has a $300 deductible and covers vision with a $200 annual cap."

Propositions extracted:

- "InstaLife Plan A has a $300 deductible."
- "InstaLife Plan A covers vision."
- "InstaLife Plan A vision coverage has a $200 annual cap."

Each proposition becomes its own chunk. This maximizes retrieval precision — every chunk is exactly one fact. The downside: LLM inference at indexing time is expensive. Worth it for high-stakes corpora where precision matters more than cost.

Late chunking

Traditional chunking: split text + embed each chunk independently. The chunk's embedding has no awareness of surrounding context.

Late chunking: embed the entire document first using a long-context embedding model, preserving full contextual awareness in every token representation. Then pool the embeddings within each chunk boundary. Each chunk's embedding reflects the full document context, not just the isolated passage.

The result: better embeddings, especially for pronouns and references that need surrounding context to be understood. Jina8192 and newer BGE models support this.

Contextual chunk headers

Anthropic released a simple but powerful technique before writing a chunk, prepend a summary of its document context. So instead of:

```
"The deductible is $300 per year."
```

You store:

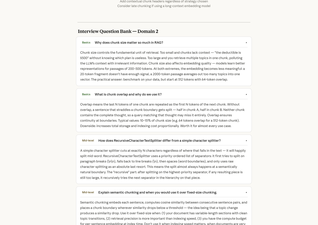
```
"Document: InstaLife Group Benefits Guide 2024, Section: Plan A Coverage Details.
The deductible is $300 per year."
```

The embedding now captures the context alongside the content. This significantly helps when the same generic phrase appears across many documents — "the deductible is $300" could be from any plan, but with the header it's unambiguous.

Parent-child chunk relationships

One of the most practically useful patterns. Index small child chunks (128 tokens) for precision retrieval. When a child chunk is retrieved, return its parent chunk (512–1024 tokens) to the LLM for context.

Small chunks = precise similarity matching. Large chunks = sufficient context for the LLM to answer well.



RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)

RAPTOR builds a multi-level tree of summaries. At the leaf level, original chunks. One level up: summaries of groups of related chunks. Another level up: summaries of summaries. At query time, you can retrieve from any level of the tree depending on whether the question needs detailed answer or a broad overview.

This elegantly solves the question "what level of detail does this query need?" — a question like "summarize the entire insurance guide" retrieves from high-level summary nodes. "What is the exact deductible for vision?" retrieves from leaf level chunks.

6. Chunk Size Trade-offs — The Calibration Science

This is where interviews test your engineering judgment. There is no universally optimal chunk size. Here's how to reason about it:

How chunk size affects key RAG metrics (conceptual, not measured. Drag to explore.)

The chart shows the core tension:

As chunk size increases: precision drops (more irrelevant content per chunk), noise rises, but context richness increases. Embedding quality peaks around 256–512 tokens and degrades for very long chunks — tiny chunks lack enough context for a meaningful vector, huge chunks average out to a fuzzy topic.

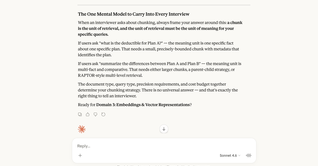
The **sweet spot is typically 256–512 tokens** for most use cases, but the right answer always comes from benchmarking on your actual data.

Benchmark-driven chunk size selection: The correct engineering approach is to set up a golden evaluation set (100–500 query-answer pairs), run retrieval at multiple chunk sizes (128, 256, 512, 1024), measure hit rate and MRR, pick the winner. Never guess.

The relationship that interviewers love asking about: **Recall vs precision.** Smaller chunks = higher precision (less noise per chunk) but lower recall (relevant information may be split across multiple chunks). Larger chunks = higher recall (more relevant information may be split across multiple chunks) but lower precision. Parent-child chunking is a direct solution: use small for retrieval precision, large for answer quality.

Chunking Strategy Decision Framework

Use this in interviews when asked "how would you choose a chunking strategy for a new project?"



Interview Question Bank — Chunking

Basic **Why does chunk size matter so much in RAG?**

Chunk size controls the fundamental unit of retrieval. Too small and chunks lack context — "the deductible is $300" without knowing which plan is useless. Too large and you retrieve mostly noise alongside the relevant fact, diluting the signal the LLM sees. Chunk size also affects embedding quality: transformer representations for passages of 200–500 tokens. At both extremes, the embedding becomes less meaningful. A 20-token fragment doesn't have enough signal, a 2000-token passage averages out too many topics into one vector. The practical answer: benchmark on your data, but start at 256 tokens with 64-token overlap.

Basic **What is chunk overlap and why do we use it?**

Overlap means the last N tokens of one chunk are repeated as the first N tokens of the next chunk. Without overlap, a sentence that straddles a chunk boundary gets split — half in chunk A, half in chunk B, neither chunk contains the complete thought, so a query matching that thought may miss it entirely. Overlap ensures continuity at split points. Typical values: 10–20% of chunk size. Trade-off: increases total chunk count (slightly) and storage cost. Worth it for almost every use case.

Mid-level **How does Recursive/Character TextSplitter decide where to split?**

A simple character splitter cuts at exactly N characters regardless of where that falls in the text — it will happily split mid-word. Recursive/character TextSplitter uses a priority ordered list of separators. It first tries to split on paragraph breaks (`"\n\n"`), then line breaks (`"\n"`), then sentence boundaries (`". "`), then spaces (`" "`), then character splitting as an absolute last resort. This means the split almost always happens at a semantically natural boundary. The "recursive" part: after splitting on the highest priority separator, if any resulting piece is still too large, it recursively tries the next separator in the hierarchy on that piece.

Mid-level **Explain semantic chunking and when you would use it over fixed-size chunking.**

Semantic chunking embeds each sentence, compares cosine similarity between consecutive sentences pairs, and places a chunk boundary wherever similarity drops below a threshold — the idea being that a topic change produces a similarity drop. Use it over fixed-size when: (1) your document has variable length sections with clear topic boundaries (e.g. a long report covering multiple unrelated topics), (2) you have budget for the extra embedding cost at indexing time, (3) fixed-size is producing noticeably worse retrieval quality. Avoid it when: documents are short, or when topics blend gradually without sharp transitions. Key implementation detail: the threshold method (percentile vs std dev vs fixed) significantly affects chunk count — always tune on a representative sample.

Mid-level **What is the parent-child chunking pattern and what problem does it solve?**

Parent-child chunking splits the tension between retrieval precision and generation quality. You create two levels of chunks: small child chunks (e.g. 128 tokens) for fine-grained similarity matching, and parent chunks (e.g. 512 tokens) stored in a docstore. During retrieval, you search the small chunks — small chunks produce more precise embeddings and match queries more exactly. When a child chunk matches, you return its parent chunk to the LLM. Concretely: a query matches Child A2 → the docstore looks up Parent A → the LLM receives Parent A's full context. The key insight: small chunks are optimized for retrieval, large chunks are optimized for generation. Parent-child chunking lets you do both simultaneously.

Senior **How would you chunk a mixed-content PDF with text, tables, and diagrams for a production RAG system?**

It's not a single chunk strategy, it's a pipeline. First, parse with OCR (e.g. Azure Document Intelligence) to extract content with layout awareness — they can separate text regions, table cells, and figure captions. For text: use recursive splitting with header breadcrumbs preserved. For tables: keep each table as one chunk (never split a table mid-row), or convert to prose chunks ("row 1: name=Alice, age=30"). For figures: generate a caption with a vision model (e.g. GPT-4o) and index the caption like a text chunk. Tag every chunk with its source page, section, and chunk type (`text`, `table`, `figure_caption`) so you can filter at query time if needed. Critical: never let a table get split across two chunks — a split table is worse than no table at all.

Senior **What is late chunking and how does it improve over traditional embedding?**

Traditional chunking: embed each chunk in isolation. Late chunking: run the entire document through a long-context embedding model (Jina8192, BGE-M3) to produce contextual token embeddings for the full document, then mean-pool the token embeddings within each chunk boundary. Each chunk's embedding now reflects its surrounding context, not just its own text. Why it matters: pronouns, references ("This plan", "the deductible") and domain-specific terms (`InstaLife`) carry their meaning. Late chunking lets the model resolve these references because it saw the full document when it created the token representations. The limitation: requires a model that supports long context embedding (Jina8192 or Jina v3 recommended, or BGE-M3). More compute at indexing time. Worth it for documents with heavy cross-referencing, like legal or medical documents.

The One Mental Model to Carry Into Every Interview

When an interviewer asks about chunking, always frame your answer around this: **a chunk is the unit of retrieval, and the unit of retrieval must be self-contained enough to answer your specific queries.**

If users ask "what is the deductible for Plan A?" — the meaning unit is one specific fact about one specific plan. That needs a small, precisely bounded chunk with metadata that identifies the plan.

If users ask "summarize the differences between Plan A and Plan B" — the meaning unit is much fuzzier and comparative. That needs either larger chunks, a parent-child strategy, or RAPTOR-style multi-level retrieval.

The document type, query type, precision requirements, and cost budget together determine your chunking strategy. There is no universal answer — and that's exactly the right thing to tell an interviewer.

Ready for **Dense & Embeddings & Vector Representations**?

Reply...

Sonnet 4.5

Claude is AI and can make mistakes. Please double-check responses.